16-bit Pipelined ALU: RTL-to-GDSII ASIC Flow using Cadence Tools

Project Overview

This project demonstrates the complete RTL-to-GDSII ASIC design flow for a 16-bit pipelined Arithmetic Logic Unit (ALU) using Cadence EDA tools in a 45nm technology node.

The design was implemented starting from Verilog RTL, verified through simulation, synthesized using Cadence Genus, and physically implemented using Cadence Innovus.

---

1. Design Specifications

Module Name

`ALU`

Technology Node

45nm GPDK

Tools Used

Stage	Tool
RTL Design	Vivado / Verilog
Functional Verification	Xcelium / NCSim
Synthesis	Cadence Genus
Physical Design	Cadence Innovus
Layout Generation	Cadence Innovus

---

2. ALU Architecture

Inputs

Signal	Width	Description
clk	1	Clock input
rst	1	Reset signal
A	16-bit	Operand A
B	16-bit	Operand B
opcode	4-bit	Operation selector
valid_in	1	Input valid signal

Outputs

Signal	Width	Description
result	16-bit	ALU output
zero	1	Zero flag
carry	1	Carry flag
overflow	1	Overflow flag
negative	1	Negative flag
valid_out	1	Output valid signal

---

3. Supported Operations

Opcode	Operation
0	Addition
1	Subtraction
2	Multiplication
3	Division
4	AND
5	OR

Opcode	Operation
6	XOR
7	NOT
8	Shift Left
9	Shift Right
10	Compare Equal
11	Compare Greater

---

4. Pipeline Architecture

The ALU was implemented using a 2-stage pipeline architecture.

Stage 1

- Input capture
- Registering operands
- Registering opcode
- Valid signal synchronization

Stage 2

- Combinational ALU computation
- Flag generation
- Output register stage

Pipeline registers improved timing and enabled synchronous operation.

---

5. Functional Verification

A dedicated Verilog testbench was developed to verify:

- Arithmetic operations
- Logical operations
- Shift operations
- Comparison operations
- Overflow conditions
- Divide-by-zero handling
- Pipeline valid signal propagation

Simulation Result

The simulation verified:

- Correct arithmetic outputs
- Correct overflow detection
- Proper flag generation
- Proper pipelined operation

The waveform analysis confirmed successful RTL functionality.

---

6. Synthesis Flow using Cadence Genus

Synthesis Steps

1. Read RTL
2. Elaborate design
3. Apply clock constraints
4. Map RTL to standard cells
5. Generate reports
6. Export synthesized netlist

Clock Constraint

Clock period used:

1000 ps (1 ns)

Equivalent operating frequency:

1 GHz

---

7. Synthesis Results

Area Report

Parameter	Value
Total Cell Count	1100
Sequential Cells	57

Parameter	Value
Combinational Cells	1043
Total Cell Area	8498.473

Observation

The synthesized design inferred optimized arithmetic structures including CSA-tree multiplier implementations.

---

8. Power Analysis

Total Power

Approximately:

2.4 mW

Power Distribution

Category	Contribution
Logic	86.9%
Registers	13.1%

Observation

The multiplier and combinational datapath dominated switching activity and power consumption.

---

9. Timing Analysis

Timing reports indicated unconstrained timing paths due to incomplete SDC/MMMC timing setup.

Current Status

- Functional synthesis successful
- Timing constraints partially defined
- Full timing closure pending

Future Improvement

- Add input/output delays
- Configure MMMC corners
- Perform proper CTS-aware timing optimization

---

10. Physical Design using Innovus

Physical Design Flow

1. Load synthesized netlist
2. Load LEF and LIB files
3. Initialize floorplan
4. Placement
5. Clock Tree Synthesis (CTS)
6. Routing
7. GDSII stream out

---

11. Technology Files Used

LEF Files

- gsclib045_tech.lef
- gsclib045_macro.lef

Liberty Timing File

- slow.lib

Technology node:

45nm GPDK

---

12. Physical Implementation Results

Placement

Standard cells were successfully placed inside the core area.

Routing

Multi-layer routing completed successfully.

The routed design showed:

- Signal routing
- Via insertion
- Metal layer interconnects
- Dense arithmetic datapath routing

---

13. GDSII Generation

Final GDSII layout generation completed successfully.

Generated files:

File	Description
final_ALU.gds	Final layout database
final_ALU.v	Final routed netlist
final_ALU.enc	Innovus database

---

14. Issues Encountered

1. SimVision Runtime Crash

The Xcelium GUI backend terminated unexpectedly during waveform viewing.

Root Cause

- Legacy Cadence environment instability
- GUI/X11 runtime issues
- SimVision backend instability

Resolution

Terminal-based simulation and backend flow continuation.

---

2. Missing Timing Constraints

Innovus reported:

- No constrained timing paths
- Missing delay corners
- Incomplete MMMC setup

Impact

- CTS optimization incomplete
- Timing-driven optimization unavailable

---

3. DRC and Connectivity Violations

Violations remained due to:

- Incomplete IO constraints
- Missing advanced physical setup
- Simplified academic backend flow

---

15. Learning Outcomes

This project provided practical exposure to:

- RTL design methodology
- Pipeline architecture
- ASIC synthesis flow
- Standard cell mapping
- Timing analysis
- Power analysis
- Floorplanning
- Placement and routing
- GDSII generation
- Backend implementation concepts

---

16. Future Improvements

RTL Enhancements

- Carry Lookahead Adder

- Wallace Tree Multiplier
- Low-power architecture
- Parameterized ALU width

Backend Enhancements

- Proper MMMC setup
- Full CTS optimization
- DRC-clean routing
- LVS verification
- STA using Tempus
- Power optimization

---

17. Conclusion

A complete RTL-to-GDSII implementation flow for a 16-bit pipelined ALU was successfully demonstrated using Cadence Genus and Innovus in 45nm technology.

The project validated the transformation of Verilog RTL into a physically routed ASIC layout and provided practical understanding of modern digital ASIC implementation flow.

The successful generation of the GDSII file marked completion of the complete backend implementation pipeline.